

Time-stamp: < 09 oct 2024 11:55 Philippe Queinnec >

Algorithme d'exclusion mutuelle à base de *jeton*.

EXTENDS *Naturals*, *FiniteSets*

CONSTANT *N*

ASSUME $N \in \text{Nat} \wedge N > 1$

$\text{Processus} \triangleq 0 \dots N - 1$

$\text{Hungry} \triangleq \text{"H"}$

$\text{Thinking} \triangleq \text{"T"}$

$\text{Eating} \triangleq \text{"E"}$

VARIABLES

etat,

jeton

$\text{TypeOK} \triangleq$

$\square(\wedge \text{etat} \in [\text{Processus} \rightarrow \{\text{Hungry}, \text{Thinking}, \text{Eating}\}]$
 $\wedge \text{jeton} \in \text{Processus})$

un processus max a le *jeton*

$\text{ExclMutuelle} \triangleq$

$\square(\forall i, j \in \text{Processus} : (\text{etat}[i] = \text{Eating} \wedge \text{etat}[j] = \text{Eating}) \Rightarrow (i = j))$

un processus qui demande obtiendra plus tard

$\text{VivaciteIndividuelle} \triangleq$

$\forall i \in \text{Processus} : (\text{etat}[i] = \text{Hungry} \leadsto \text{etat}[i] = \text{Eating})$

si un processus demande, un processus obtiendra plus tard (pas forcément le même)

$\text{VivaciteGlobale} \triangleq$

$(\exists i \in \text{Processus} : \text{etat}[i] = \text{Hungry}) \leadsto (\exists j \in \text{Processus} : \text{etat}[j] = \text{Eating})$

$\text{JetonVaPartout} \triangleq$

$\forall i \in \text{Processus} : \square \diamond (\text{jeton} = i)$

$\text{Sanity} \triangleq$

$\square(\forall i \in \text{Processus} : \text{etat}[i] = \text{Eating} \Rightarrow \text{jeton} = i)$

$\text{Init} \triangleq$

$\wedge \text{etat} = [i \in \text{Processus} \mapsto \text{Thinking}]$

$\wedge \text{jeton} \in \text{Processus}$

$\text{demander}(i) \triangleq$

$$\begin{aligned}
& \wedge \text{etat}[i] = \text{Thinking} \\
& \wedge \text{etat}' = [\text{etat} \text{ EXCEPT } ![i] = \text{Hungry}] \\
& \wedge \text{UNCHANGED } \text{jeton} \\
\\
\text{entrer}(i) & \triangleq \\
& \wedge \text{etat}[i] = \text{Hungry} \\
& \wedge \text{jeton} = i \\
& \wedge \text{etat}' = [\text{etat} \text{ EXCEPT } ![i] = \text{Eating}] \\
& \wedge \text{UNCHANGED } \text{jeton} \\
\\
\text{sortir}(i) & \triangleq \\
& \wedge \text{etat}[i] = \text{Eating} \\
& \wedge \text{etat}' = [\text{etat} \text{ EXCEPT } ![i] = \text{Thinking}] \\
& \wedge \text{jeton}' = (i + 1) \% N \quad \text{ajout pour supprimer } SF(\text{bouger}(i)) \\
& \wedge \text{UNCHANGED } \text{jeton} \setminus * \text{ modif pour supprimer } SF(\text{bouger}(i)) \\
\\
\text{bouger}(i) & \triangleq \\
& \wedge \text{jeton} = i \\
& \wedge \text{etat}[i] \neq \text{Eating} \\
& \wedge \text{jeton}' = (i + 1) \% N \\
& \wedge \text{etat}[i] \neq \text{Hungry} \quad \text{ajout pour supprimer } SF(\text{entrer}(i)) \\
& \wedge \text{UNCHANGED } \text{etat} \\
\\
\text{Next} & \triangleq \\
& \exists i \in \text{Processus} : \\
& \quad \vee \text{demander}(i) \\
& \quad \vee \text{entrer}(i) \\
& \quad \vee \text{sortir}(i) \\
& \quad \vee \text{bouger}(i) \\
\\
\text{Fairness} & \triangleq \forall i \in \text{Processus} : \\
& \quad \wedge \text{WF}_{\langle \text{etat}, \text{jeton} \rangle}(\text{entrer}(i)) \\
& \quad \wedge \text{WF}_{\langle \text{etat}, \text{jeton} \rangle}(\text{sortir}(i)) \\
& \quad \wedge \text{WF}_{\langle \text{etat}, \text{jeton} \rangle}(\text{bouger}(i)) \\
\\
\text{Spec} & \triangleq \\
& \wedge \text{Init} \\
& \wedge \Box[\text{Next}]_{\langle \text{etat}, \text{jeton} \rangle} \\
& \wedge \text{Fairness}
\end{aligned}$$